Portfolio Project — Summary

OTT Subscription Churn & Revenue Impact Analysis

Overview

This project is an end-to-end churn analytics pipeline built on an OTT (Over-The-Top streaming) subscription dataset. Raw customer, subscription, and support-ticket data stored across three SQLite tables was extracted, cleaned, joined, and transformed into 20+ business KPIs using Python (pandas & numpy), then visualized with matplotlib and seaborn to surface churn drivers and quantify their revenue impact.

The analysis identified an overall churn rate of **28.6%**, and found that monthly-contract subscribers churned at **55.6%** — nearly **6.7×** the **8.3%** churn rate of annual-contract subscribers. This directly ties **$73.94/month** in MRR (Monthly Recurring Revenue) leakage and **$2,047** in CLTV (Customer Lifetime Value) erosion to six at-risk customers, informing a targeted contract-migration retention strategy.

Objective

- Consolidate customer, subscription, and support data from a normalized SQLite database into a single analysis-ready dataset.
- Clean and standardize inconsistent fields (missing countries, gender label variants, duplicate support tickets).
- Engineer churn-related KPIs: churn rate, retention rate, ARPU, tenure, revenue at risk, escalation rate, and a churn-risk tier.
- Visualize churn trends over time, by plan type, and correlations between churn drivers.
- Translate the findings into a concrete, revenue-linked retention recommendation.

Dataset & Tools

Source	SQLite database (customer_churn.db) with 3 tables
Tables	db_customer, db_subscription, db_support
Records	21 customers after merge/dedup
Languages / Libraries	Python, pandas, numpy, seaborn, matplotlib, sqlite3
Environment	Jupyter Notebook

Methodology

1. Data Extraction & Loading

Connected to the SQLite database, programmatically read the table list from sqlite_master, and loaded each table (db_customer, db_subscription, db_support) into its own pandas DataFrame.

In [5]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import sqlite3
```

In [6]:
```
conn=sqlite3.connect("customer_churn.db")

sql_query="""
            SELECT name
            From sqlite_master
            Where type="table"
"""
tables=pd.read_sql(sql_query,conn)

for table_name in tables['name']:
    df=pd.read_sql(f"SELECT * FROM {table_name}",conn)
    globals()[f"df_{table_name}"]=df
    print(f"Created Dataframe:df_{table_name}")
conn.close()
```

```
Created Dataframe:df_db_customer
Created Dataframe:df_db_subscription
Created Dataframe:df_db_support
```

In [7]:
```
#for printing table name and column name

conn=sqlite3.connect('customer_churn.db')
```

Loading tables from SQLite and inspecting their schemas

2. Cleaning & Feature Engineering

Key cleaning steps included: renaming columns for clarity, dropping mostly-empty columns (interests, pincode), converting date fields to datetime, standardizing gender labels (Men/Women → Male/Female), and imputing missing country values from a state-to-country lookup built from complete rows. Support tickets were de-duplicated to one row per customer using a complaint_count derived via groupby, keeping the record with the fewest duplicates. A churn_flag was engineered from cancellation_date (1 if cancelled, 0 if still active), and the three cleaned tables were merged on customerid into a single 21-row, 23-column analysis dataset.

3. Core KPI Calculations

With the merged dataset in place, churn rate, retention rate, churn rate by plan type, and ARPU (Average Revenue Per User) were computed directly from the churn_flag and monthly_charges fields.

```
       'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
       'churn_flag'],
      dtype='object')
```

```
In [44]: print('churn_flag' in df.columns)
```

```
True
```

```
In [45]: df = (
             df_db_subscription
             .merge(df_db_customer, on='customerid', how='left')
             .merge(df_db_support, on='customerid', how='left')
         )
```

```
In [46]: df["churn_flag"].mean()
```

```
Out[46]: np.float64(0.2857142857142857)
```

```
In [47]: churn_rate=df_db_subscription["churn_flag"].mean()*100
         print("churn_rate= ",round(churn_rate,2),"%")
```

```
churn_rate=  28.57 %
```

```
In [48]: retention_rate=100-churn_rate
         print("retention_rate= ",round(retention_rate,2),"%")
```

```
retention_rate=  71.43 %
```

```
In [49]: df.groupby('plan_type')['churn_flag'].mean()
```

```
Out[49]: plan_type
         Basic       0.600000
         Premium     0.142857
         Standard    0.222222
         Name: churn_flag, dtype: float64
```

Churn rate, retention rate, and churn rate by plan type

4. Revenue & Tenure Metrics

Tenure was calculated in days for every customer — using the gap between subscription start and cancellation for churned users, and start date to today for active users — giving an average tenure of roughly 1,504 days. Revenue at risk was calculated as the sum of monthly_charges for every customer flagged as churned.

	MKNFE	2020-08-01	Paid	2024-08-01	Premium
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard

5 rows × 24 columns

In [54]:
```
avg_tenure=df['tenure_days'].mean()
print(avg_tenure)
```

```
1503.7142857142858
```

In [55]:
```
#revenue at risk
revenue_at_risk=df.loc[df['churn_flag']==1,'monthly_charges'].sum()
print(revenue_at_risk)
```

```
73.94
```

Tenure-in-days feature and revenue-at-risk calculation

5. Churn Risk Segmentation

Customers were bucketed into low / medium / high churn-risk tiers using numpy.select against churn_score thresholds, creating a simple, business-readable risk flag for prioritizing outreach.

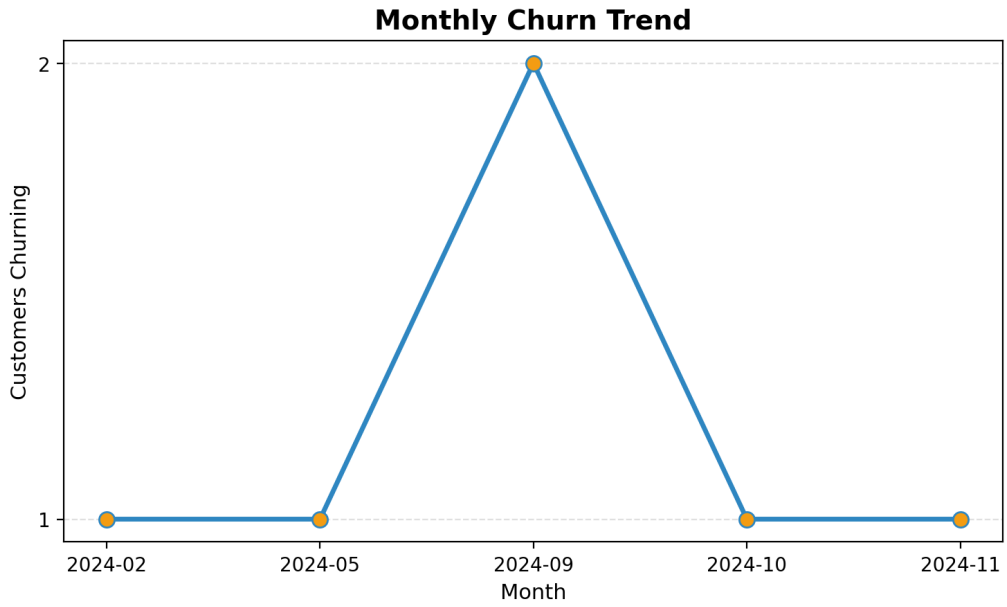
```
df[['churn_score', 'churn_risk']].head(10)
```

Rule-based churn_risk tier built from churn_score

Visualizations

Monthly Churn Trend

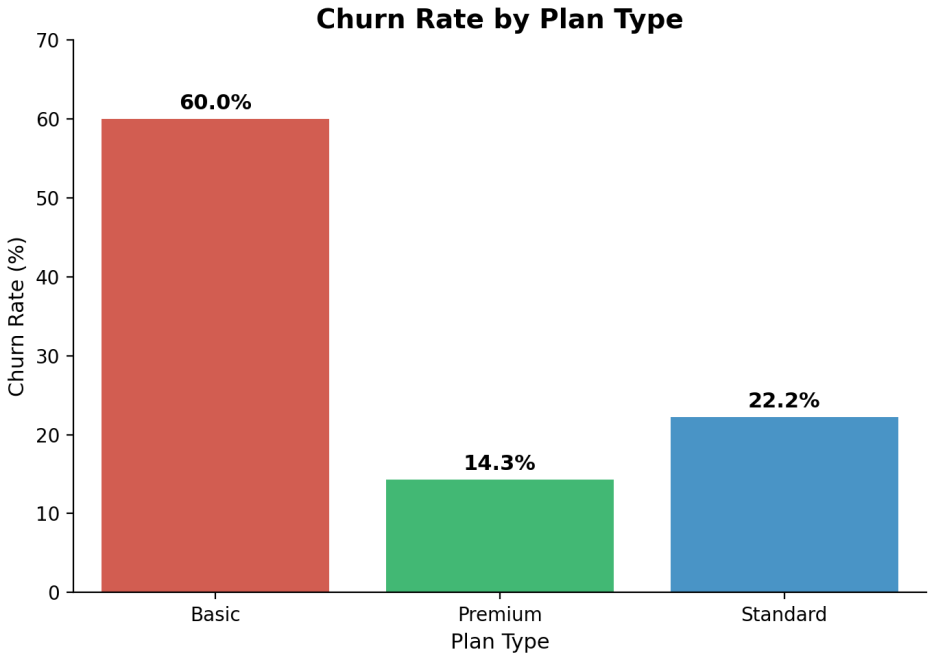
Cancellations were grouped by month to trace churn volume over time, highlighting a spike in September 2024 that warrants further root-cause investigation.



Number of customers churning per month

Churn Rate by Plan Type

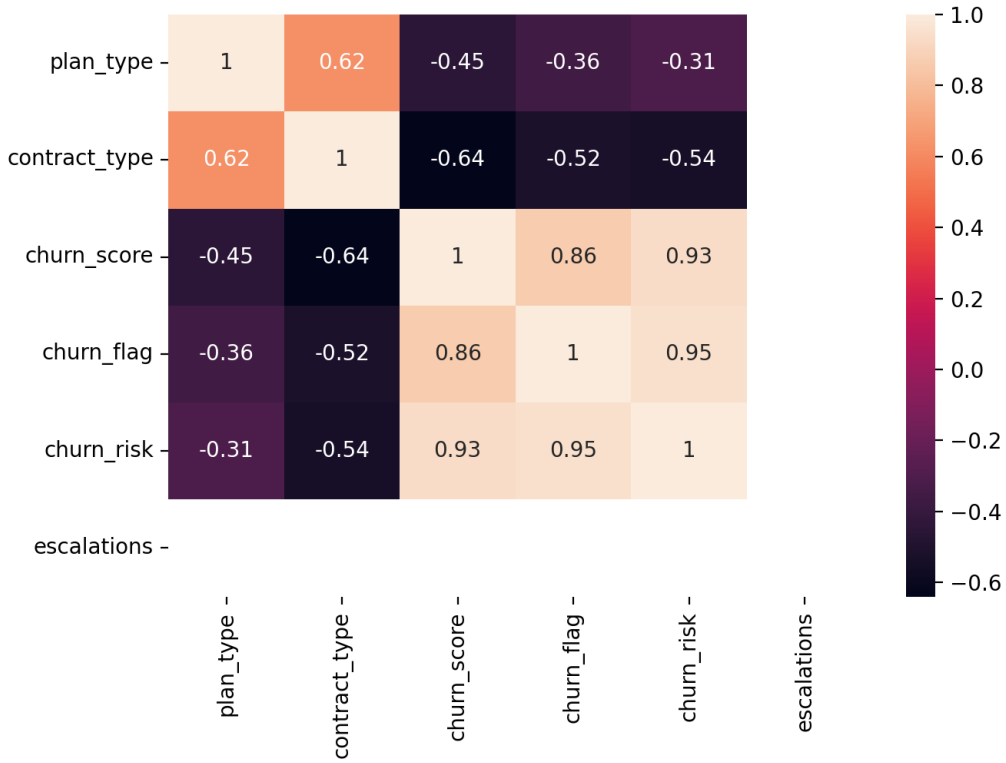
Basic-plan subscribers churn at roughly 60%, more than 4× the Premium plan's 14.3% rate, suggesting the Basic tier under-delivers on perceived value relative to price.



Churn rate segmented by subscription plan type

Correlation Heatmap

A correlation matrix across plan_type, contract_type, churn_score, churn_flag, and churn_risk shows churn_score and churn_risk are very strongly correlated with actual churn (0.86 and 0.95 respectively), and contract_type has a meaningful negative correlation with churn (-0.52), reinforcing that annual contracts are protective against churn.



Correlation between plan/contract attributes and churn outcomes

Key Findings

Metric	Value
Overall churn rate	28.6%
Retention rate	71.4%
Monthly-contract churn rate	55.6%
Annual-contract churn rate	8.3%
Basic plan churn rate	60.0%
Premium plan churn rate	14.3%
ARPU	$18.85
Average tenure	~1,504 days
Revenue at risk (MRR)	$73.94 / month
CLTV erosion (6 at-risk customers)	$2,047
Escalation rate	19.05%
Avg. complaints per customer	0.43

Business Recommendation

Because monthly-contract subscribers churn at nearly 7× the rate of annual-contract subscribers, and Basic-plan users churn at over 4× the rate of Premium users, the highest-leverage retention lever is a **targeted contract-migration campaign**: proactively offer discounted annual-contract upgrades to high churn-risk, monthly-billed Basic/Standard customers identified via the churn_risk tier, before they reach their renewal date. Given the $73.94/month MRR and $2,047 CLTV currently at risk from just six customers, converting even a fraction of the at-risk monthly base to annual contracts would materially reduce revenue leakage.

Tech Stack

- Python 3 — pandas, numpy, seaborn, matplotlib, sqlite3
- Jupyter Notebook for exploratory analysis
- SQLite as the source database (3 normalized tables)

This README was generated from the project notebook (Churn_analysis.ipynb / .pdf export), combining narrative summary, notebook code excerpts, and the notebook's own matplotlib/seaborn visualizations.